

# 強化学習ライブラリ ANET

2026/5/29

# 目次

## 1. 強化学習の概要

- 1.1. 強化学習とは
- 1.2. 強化学習の立ち位置
- 1.3. LLMと強化学習

## 2. ANETの概要

- 2.1. 強化学習とは
- 2.2. ANETの概要
- 2.3. ソフトウェア構成
- 2.4. 機能概要
- 2.5. 開発プロセス
- 2.6. AIツール活用

## 3. 動作イメージ: DropMerge

- 3.1. ENVモデル
- 3.2. 概念フロー
- 3.3. NetwokModel
- 3.4. 学習結果

## 4. 現状と今後

- 4.1. 現状分析
- 4.2. 今後の予定

# 強化学習の概要

# 強化学習とは

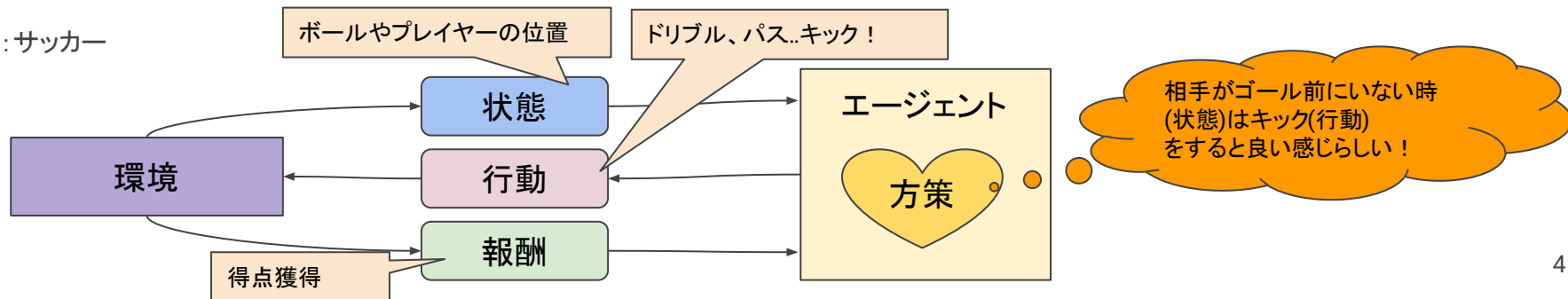
- 強化学習は機械学習の一分野です。環境との相互作用を通じて、報酬が大きくなる行動方針を学びます。
- 予め「正解」を用意できない課題でも、結果の良し悪しを教える事で学習が可能です。

## ■機械学習の分類

|          |                                                  |
|----------|--------------------------------------------------|
| 教師あり学習   | 入力データとその正解データを同時に入力し、それらの関係性を学習。                 |
| 教師なし学習   | 正解ラベルのないデータから、潜在的な構造やパターンを抽出。                    |
| 自己教師あり学習 | 入力データを自ら加工して“正解”を定義する事で教師あり学習の学習データを生成。          |
| 強化学習     | 正解ではなく「結果の良し悪し」を与える事で最適方策を学習。試行錯誤を通じて自ら学習データを生成。 |

## ■強化学習による学習イメージ

例：サッカー



# 強化学習の立ち位置

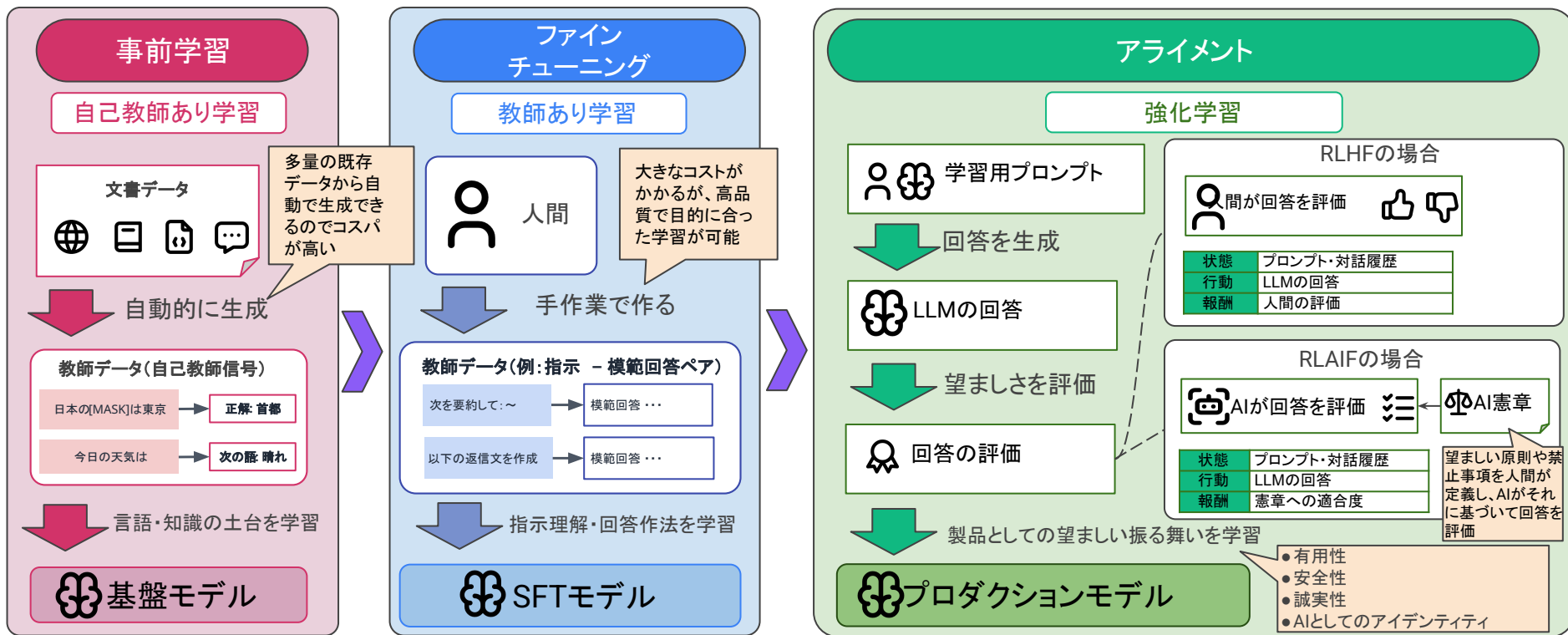
- 強化学習は応用分野によっては実用性や性能確保の為に必須要素となります。
- 自己教師あり学習など他の枠組みと組み合わせで適用されるケースが増えています。

## ■機械学習の応用分類と強化学習の位置付け

| 分類          | 具体例                       | 強化学習との関わり                                                          |
|-------------|---------------------------|--------------------------------------------------------------------|
| 生成AI        | LLM、画像生成、AIエージェント         | 主に自己教師あり学習で学習後、強化学習(RLHF等)により人間が考えるシステムの望ましい挙動になるよう調整。             |
| 認識AI        | 画像認識、音声認識、物体検出、文書分類       | 主に教師あり学習が中心。強化学習は注視領域の選択や検査手順の最適化などで活用される場合がある。                    |
| 自動制御        | 自動運転、ロボット制御               | 状態に応じてどの操作を行うかの方策を学習する。実運用では安全性のため、シミュレーション・安全制約・従来制御と組み合わせる場合が多い。 |
| ゲームAI       | 囲碁・将棋AI、ゲーム攻略             | 自己対戦を繰り返すことで、人間のトッププレイヤーを凌駕する高度な方策を自律的に獲得。(例: AlphaGo)             |
| パーソナライゼーション | リコメンド、検索レーティング、広告配信最適化    | クリック、購買、継続利用などを報酬として表示内容や配信順序を最適化。                                 |
| 最適化問題       | ロジスティクス、金融、資源配置、スケジュール最適化 | コスト、利益、待ち時間などを報酬として、継続的な意思決定を最適化。                                  |

# LLMと強化学習

- LLMの学習では、自己教師あり学習・教師あり学習に加えて、望ましい挙動に導くための強化学習「RLHF」「RLAIF」の適用が一般的です。



# ANET概要

# ANETの概要

- C++で実装された強化学習のソフトウェアフレームワークを継続開発中です。
- 現状では個人的な実験コードとしての位置付けになります。

| 項目        | 仕様内容                    | 備考                                       |
|-----------|-------------------------|------------------------------------------|
| 名称        | ANET                    | 学生時代に適当に考えた名前                            |
| 開発開始時期    | 2025年10月より開発開始          |                                          |
| 開発言語      | C++、Java(MetricsViewer) | VisualStudio 2022、C++20                  |
| 動作OS      | Windows 11              | 作りはクロスプラットフォームなので恐らくLinux上でも動作可能(試していない) |
| 主要依存ライブラリ | LibTorch、wxWidgets      |                                          |
| 計算基盤      | x64、NVIDIA CUDA         | Geforce RTX 5090で開発中                     |
| 構成管理      | GitHub                  | 全ソースコードを公開                               |
| ライセンス     | Apache2.0ベース(予定)        | 整備中                                      |
| 実装規模      | 総ステップ数約41K、実ステップ数約29K   |                                          |



# ソフトウェア構成

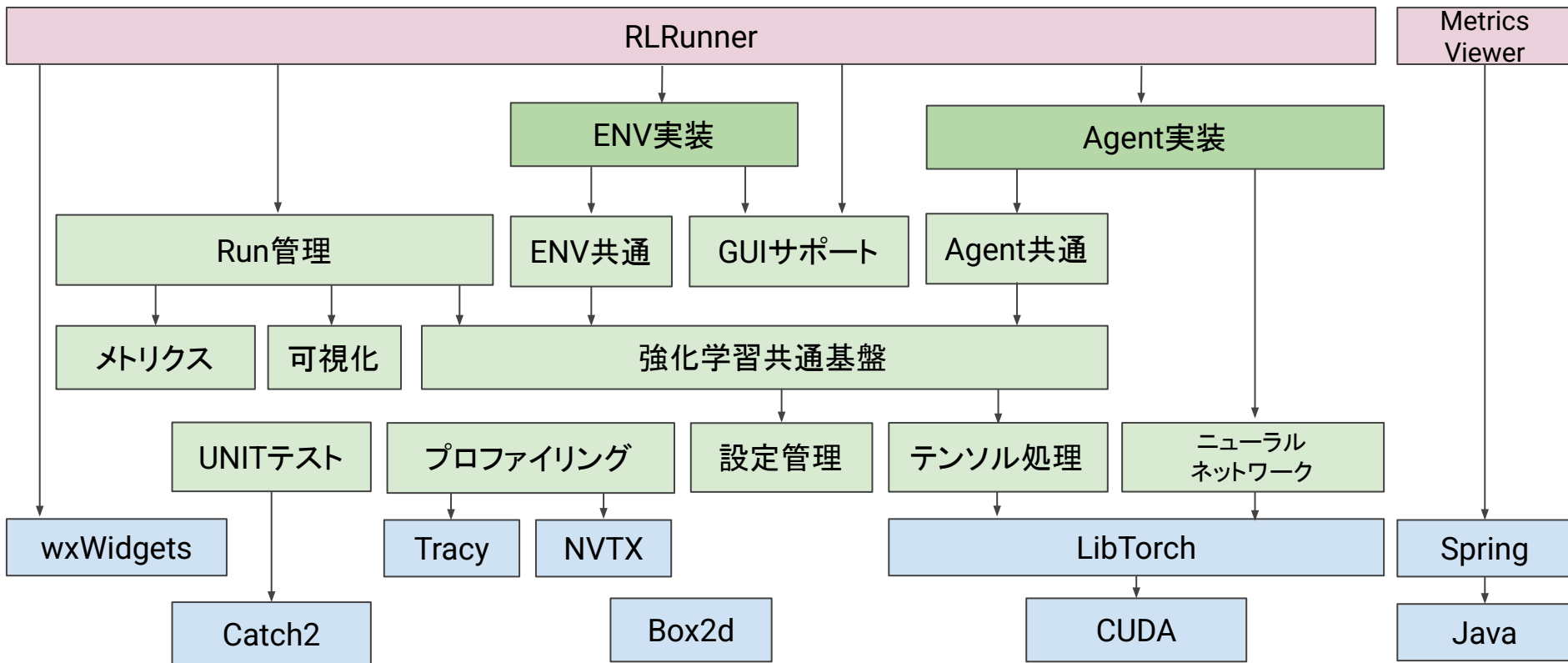
具象クラス

実行可能アプリケーション

コアライブラリ

サードパーティー・ソフトウェア

- 強化学習の学習と推論をGUIアプリとして実行可能にする部品を揃えています。



# 依存する外部ソフトウェア

- LibTorchやwxWidgetsを初めとした外部ソフトウェアを活用しています。

| 外部ソフトウェア名称 | 概要                                                              | 用途                      |
|------------|-----------------------------------------------------------------|-------------------------|
| LibTorch   | テンソル処理(自動微分など)やニューラルネットワークの基本部品を含む。機械学習で広く使われているPyTorchのバックエンド。 | テンソル計算、自動微分、ニューラルネットワーク |
| CUDA       | LibTorchのバックエンドとして動作するNVIDIAのGPU処理ライブラリ。                        |                         |
| wxWidgets  | クロスプラットフォーム対応のGUIライブラリ。                                         | GUIアプリケーション、画像処理        |
| NVTX       | 「NVIDIA Nsight」にてGPUを含む処理ボトルネックを可視化するプロファイリング用ライブラリ。            | プロファイリング、性能分析           |
| Tracy      | 実行速度やメモリ使用状況をリアルタイムにミリ秒・マイクロ秒単位で監視・可視化できる非常に強力なプロファイラー。         |                         |
| Box2d      | 2次元の物理挙動(剛体、衝突判定、接触、摩擦などをシミュレーションする定番ライブラリ。                     | 2次元物理シミュレーションを利用したENV実装 |
| Catch2     | C++向けのモダンで使いやすい単体(ユニット)テスト用フレームワーク                              | ユニットテスト                 |

# ANET 機能概要 – コアライブラリ

- 強化学習の学習と推論に必要な各種機能を提供します。

| 機能名      | 機能概要                                                                                            |
|----------|-------------------------------------------------------------------------------------------------|
| テンソル処理   | LibTorchのテンソル処理を補完。形状チェック、TensorDictを含む                                                         |
| プロファイリング | NVTXやTracyを利用した性能分析用のタグ埋め込み。スレッド別での処理タイムラインやホットスポットを可視化。NVIDIA Nsightを利用する事でCPUとGPUの統合的な可視化が可能。 |
| ユニットテスト  | Catch2を利用した単体テスト。                                                                               |
| 設定管理     | 独自のProperties(Java)類似記法設定ファイルによる設定情報管理。継承やincludeの記法にて保守性を確保。                                   |
| 可視化      | ニューラルネットワークや学習・推論状況の可視化。ヒートマップ動画、ヒストグラム動画、グラフ(ネットワーク図、dot言語ベース)、CNN活性状況など。                      |
| GUIサポート  | GUIアプリケーション実装のサポートクラス。ENVの可視化に対応。                                                               |
| メトリクス管理  | 大量のメトリクスデータをRun×タグの単位で記録・閲覧。<br>TensorBoardおよび独自のMetricsViewerにてメトリクスの確認・分析が可能。                 |

# ANET 機能概要 – コアライブラリ

- 強化学習の学習と推論に必要な各種機能を提供します。

| 機能名         | 機能概要                                                                                   |
|-------------|----------------------------------------------------------------------------------------|
| 強化学習共通基盤    | 環境インターフェース(ENV)、エージェントインターフェース(Agent)、状態(State)、行動(Action)など、強化学習で必要となる各種基本クラスを提供。     |
| ニューラルネットワーク | 設定ベースでネットワーク構成を容易に定義可能なニューラルネットワーク実装。                                                  |
| AGENT共通     | AGENT基底クラスを初めとして、ReplayBuffer、ObservationNormalizer、RewardScalerなど、Agent実装に必要な共通部品を提供。 |
| ENV共通       | ENVをスレッドプール上で高効率に動作させるための各種共通部品を提供。                                                    |
| Run管理       | AgentとEnvを駆動するRunの基本機能を提供。TrainRunner(学習用)とEvalRunner(評価用)など、複数のRunnerを並行して実行管理。       |

# ANET 機能概要 — ENV実装

- 高速動作可能なENV実装が含まれます。

| ENV名        | 概要                                                                                               | 備考                                        |
|-------------|--------------------------------------------------------------------------------------------------|-------------------------------------------|
| CartPole    | カートを左右に動かすことで、上に乗っているポールが倒れないようにバランスを取り、直立した状態をできるだけ長く維持する。ベクトルベースの状態空間。                         | 強化学習のアルゴリズムをテスト・評価するための最も代表的なベンチマーク環境の1つ。 |
| LunarLander | 月面着陸船シミュレーション。Box2dによる物理演算を適用。3か所のエンジンを噴射し、着陸パッドに正しい姿勢で着陸する事が目的となる。風と地形というランダム要素あり。ベクトルベースの状態空間。 | Gymnasium版を参考とした再実装。                      |
| DropMerge   | スイカゲームLike。画像ベースの状態情報。                                                                           | 詳細は後述                                     |
| GridMaze    | シンプルな迷路問題。スタート・ゴールと落とし穴が格子状の空間上に生成される。盤面の情報は直接見えないため、探索が必要。ゴールするまで報酬が発生しない。ベクトルベースの状態空間。         | 基本的にはアルゴリズム動作確認用。                         |

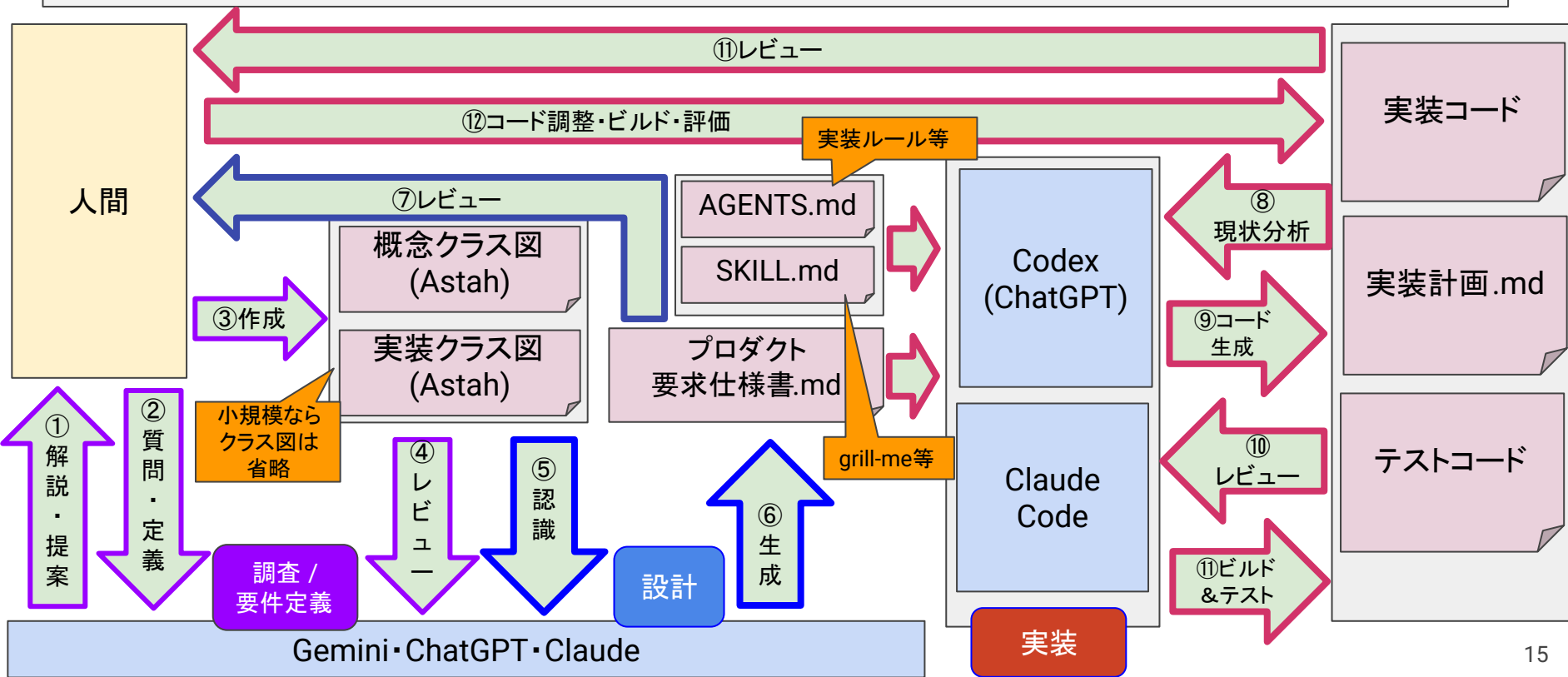
# ANET 機能概要 — Agent実装

- DQN系列のアルゴリズム実装をメインとしています。
- 現状では数が少ないですが、順次拡大予定です。

| Agentアルゴリズム | 概要                                                                                                                                                                                                                                         | 備考                      |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| DQNBased    | 価値ベースRLアルゴリズムの代表格、DQN(Deep Q-Network) 系列のエージェント実装。Rainbowの構成要素に加えて、設定次第でQR-DQNなどの一部後続アルゴリズムに対応。 <ul style="list-style-type: none"><li>● Double-DQN</li><li>● PER</li><li>● N-STEP Learning</li><li>● DuelingNet</li><li>● QR-DQN</li></ul> | C51、NoisyNetには非対応       |
| MuZero(試作版) | モデルベースの強化学習アルゴリズム「MuZero」の試作版。オリジナル論文ベース、決定論的環境にのみ対応。分布学習無し、PER非対応、Reanalyze非対応。                                                                                                                                                           | 試作版のため、学習可能ながら動作は非常に遅い。 |
| ImageCls    | 画像分類問題を学習。正確には強化学習ではなく教師あり学習。                                                                                                                                                                                                              | ハイパラ探索は未実施。             |

# 開発プロセス

- ANETの開発はAIエージェントを活用しています。



# AIツール活用

- 3種のLLMを適材適所で活用しています。

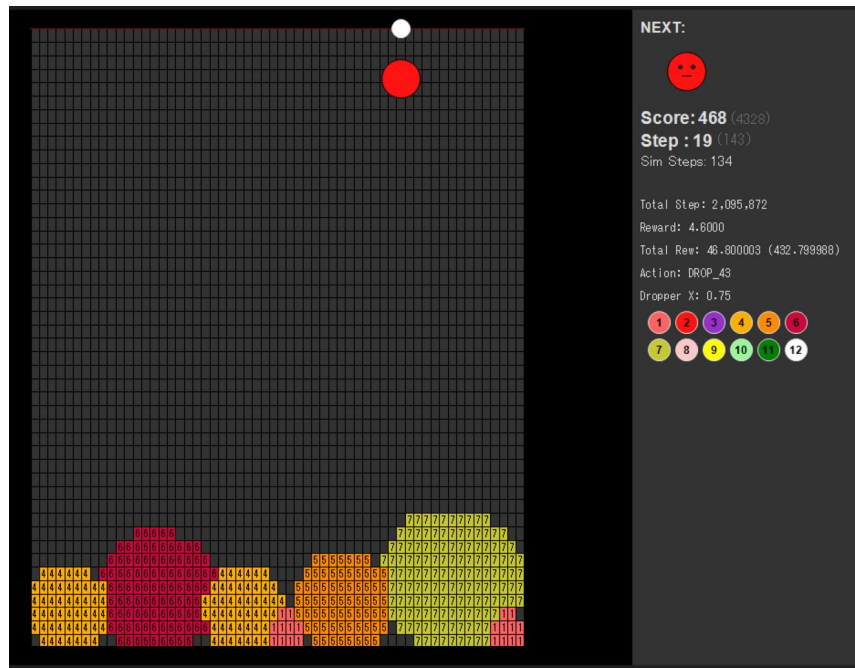
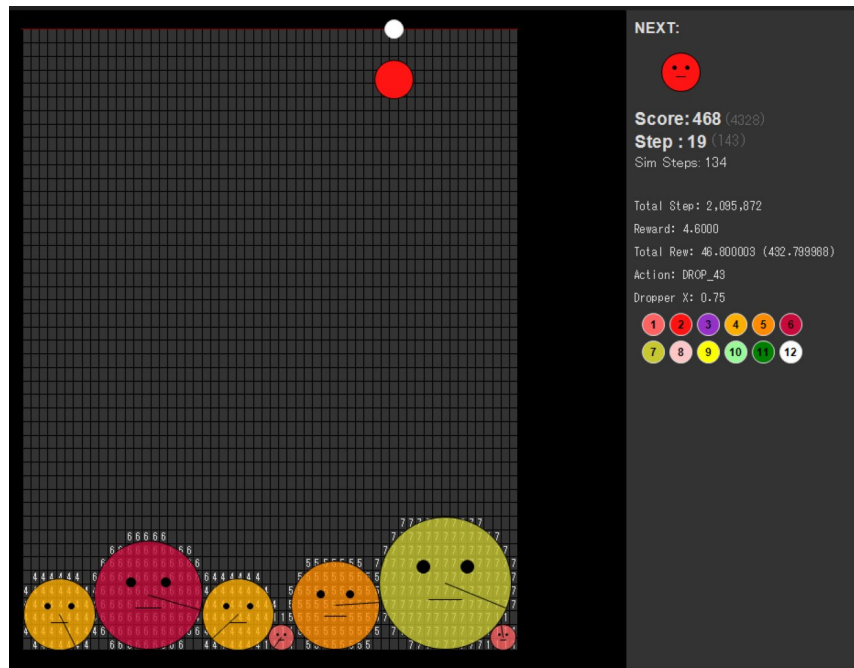
| サービス               | 長所                                   | 短所                                 | ANET開発における用途                                                                                                                        |
|--------------------|--------------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Gemini             | 長大なコンテキスト容量、<br>Google基盤との連携         | AIエージェント機能が弱い<br>(Antigravityは未評価) | <ul style="list-style-type: none"><li>● 調査の壁打ち</li><li>● 外部仕様の調整</li><li>● 内部仕様の調整</li><li>● ハイパラ調整</li><li>● 特定トピックの長期議論</li></ul> |
| Codex<br>(ChatGPT) | ソースを自発的に見て分析し、<br>修正までほぼ自動。          | ちょっと処理が遅い(設定次第?)                   | <ul style="list-style-type: none"><li>● 複数ソースを意識した分析</li><li>● 内部仕様の調整</li><li>● コード生成</li></ul>                                    |
| Claude Code        | 基本はCodexと同じ。加えて<br>矛盾検知やコーディング能力が高い。 | 価格が高い。プラン次第ながらすぐ<br>トークン上限に達する。    | <ul style="list-style-type: none"><li>● 各種レビュー</li><li>● 難しい課題や不具合の相談</li></ul>                                                     |



# 動作イメージ: DropMerge

# DropMerge: ENVモデル

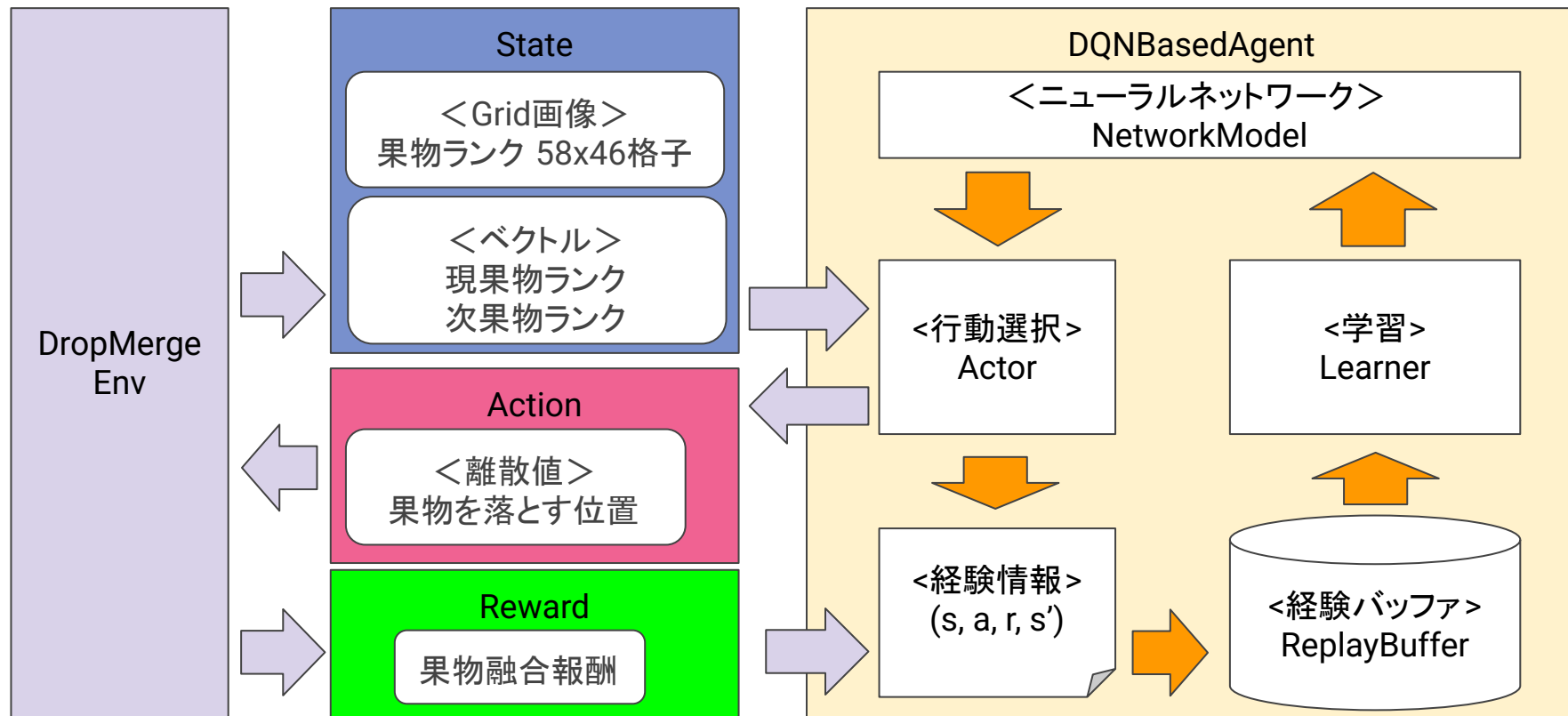
- スイカゲームLikeな独自ENVです。Box2dにて物理演算を行っています。
- 同じ種類(ランク)の果物2つが接触すると一つ上のランクの果物1つに変化します。
- 物理演算が入る為に複雑性が高め、かつ次果物がランダムであり、長期的戦略も必要となります。



※物性(大きさ、重さ、摩擦など)は本家と完全にはありません。

# DropMerge: 概念フロー

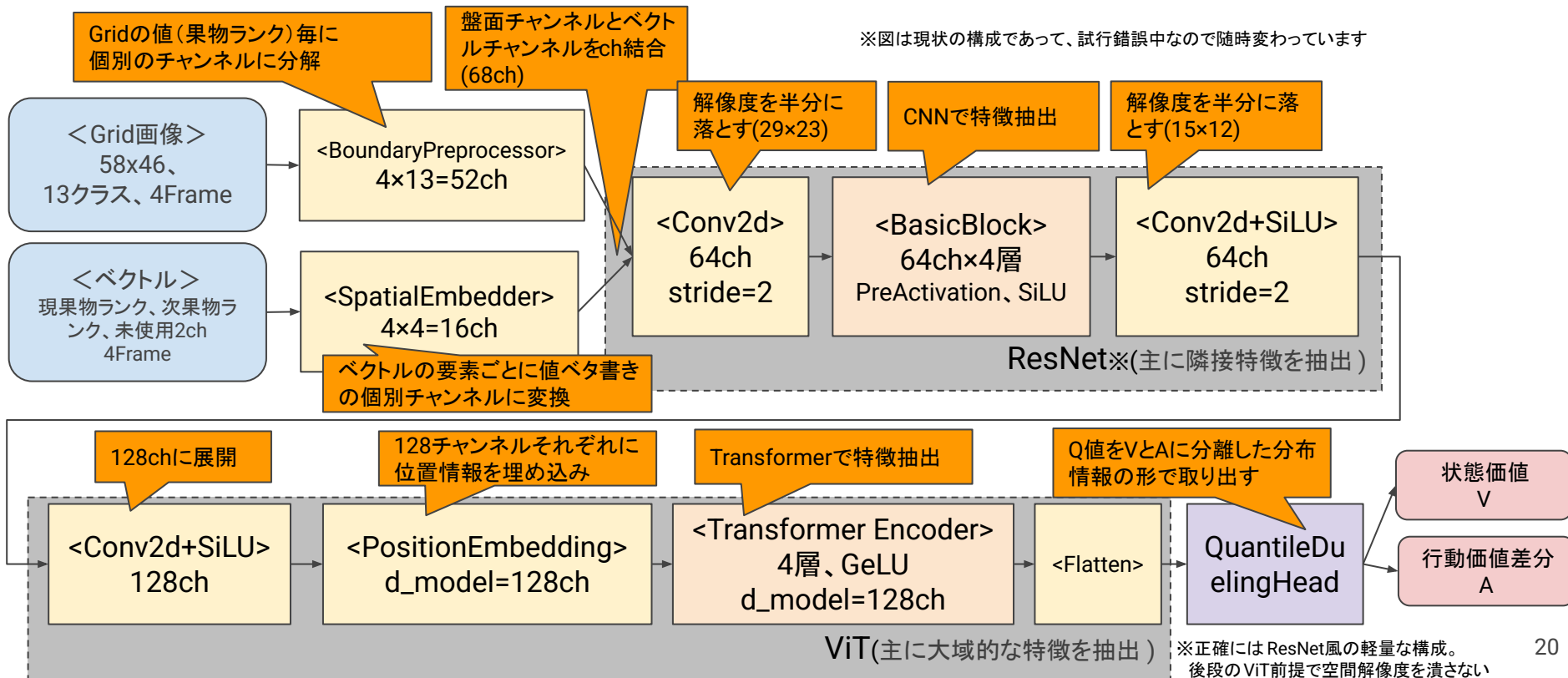
- Rainbow+QR-DQNアルゴリズムで、盤面を画像認識して学習します。



# DropMerge : NetworkModel

- ResNet+ViTでハイブリッド構成のニューラルネットワークを採用しています。

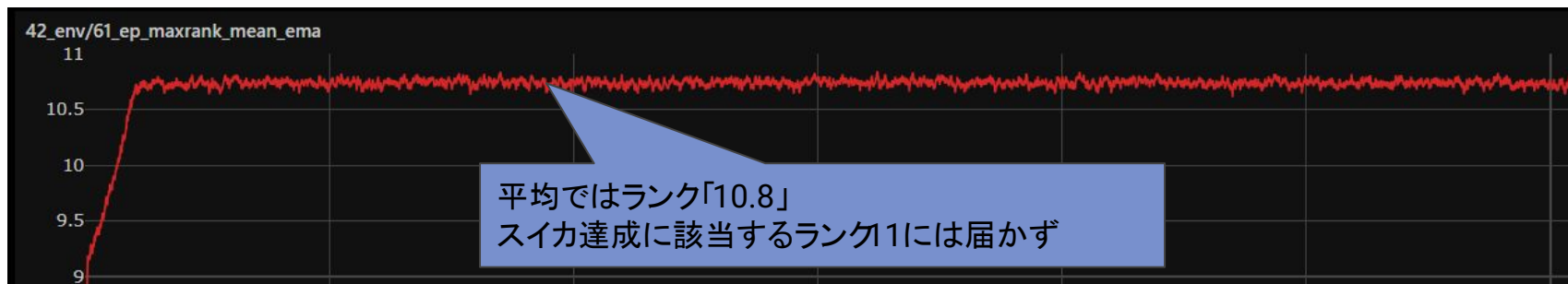
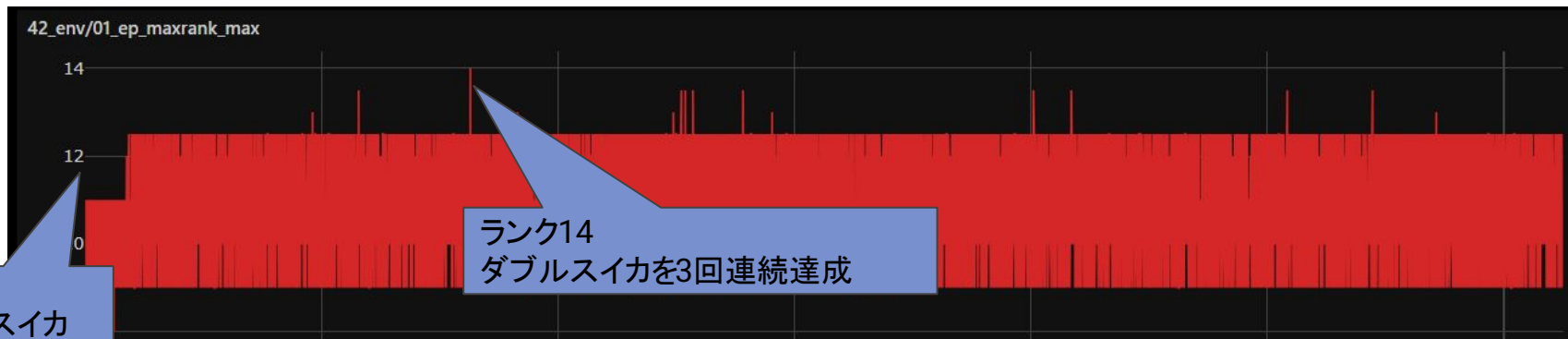
※図は現状の構成であって、試行錯誤中なので随時変わっています



# DropMerge: 学習結果

- 約5日間に渡って学習させた結果、人間でも容易ではないダブルスイカを高頻度で達成出来ています。
- ただし平均ランクは10.8、つまり「ほぼスイカ」どまり。改善の余地があります。

## ■約1600Mステップ学習後



# 現状と今後

# 現状分析(SWOT)

- C++でGUI・可視化・実験実行まで統合している点が強み。
- 一方で、個人開発である事による絶対的なリソース不足が課題となっています。

## STRENGTHS (強み)



- C++とプロファイリングによる高速性
- 柔軟なニューラルネットワーク定義
- 学習過程の可視化/分析の基盤が豊富

## WEAKNESSES (弱み)



- ドキュメントが整備されていない
- コメントや文書が日本語Only
- 学習アルゴリズムのバリエーションが少ない
- UNITテストコードが不足
- 随時開発しているため、API安定性が低い

## OPPORTUNITIES (機会)



- C++で統合的なRLフレームワークは世界的に見ても事例が少ない
- テンプレート利用を必要最小限としており、クラス構成の面でも可読性が高め

## THREATS (脅威)



- Python一強の中でC++やWindowsに依存している為、利用者適用性が低い
- C++の強みである組込みシステムへの応用性をWindowsやLibTorch依存によって生かせない

# 今後の予定

- 個人の実験場という位置付けは維持しつつ、OSSライブラリとしての展開を見据えて活動を継続しています。

個人のアルゴリズム探索の場という位置付けは継続予定です。一方で、長期的にはOSSライブラリとしての展開も可能性があります。

現状分析の結果も踏まえて、以下を重点として活動継続予定です。

- ドキュメント整備
- テストコード整備
- MuZero等のより新しいアルゴリズムへの対応
- Python連携による応用性向上（豊富なPython製ENV実装を活用）
- Linuxサポート
- 独自アルゴリズムの提案

ご興味あれば、評価・Issue起票・Forkなど、大歓迎です！